



---

**Análisis data**

**NYC & limousine  
commission**

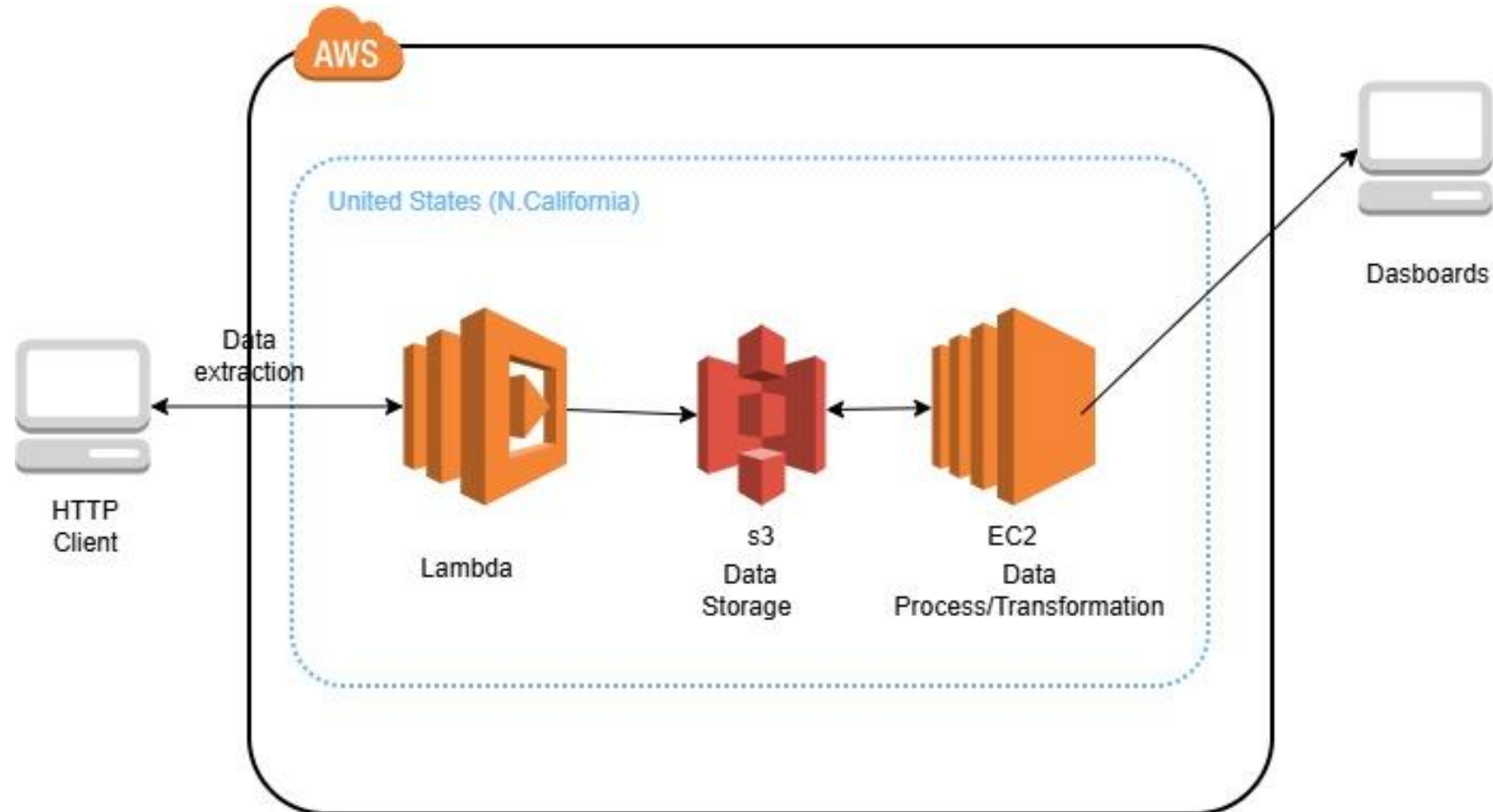
**Yellow & Green  
Taxis**

*Proyecto Integrador*

*Nancy Paola Esquivel Prado*

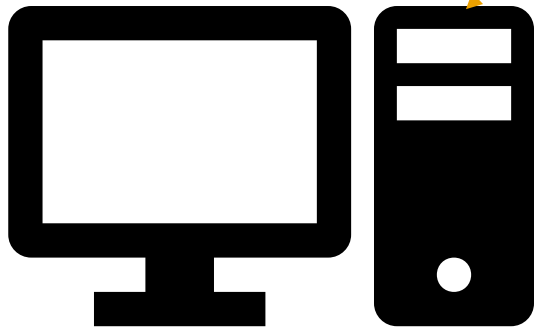
---

# Diagrama de arquitectura pipeline

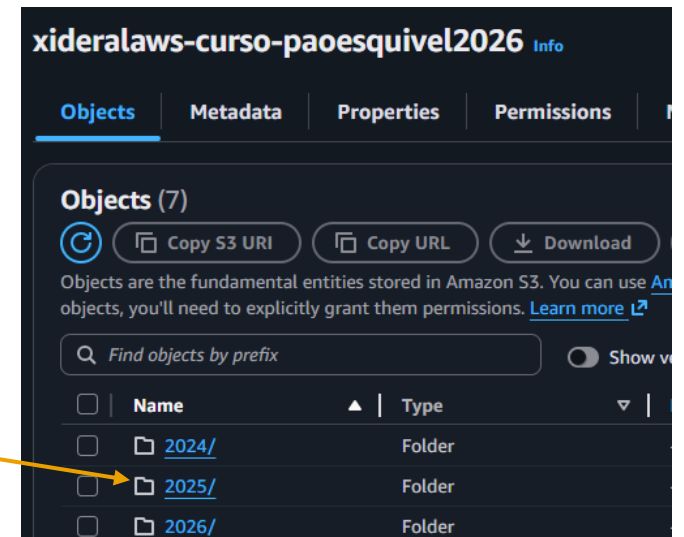


# Extracción Data

*Lambda extrae los parquets por año y los almacena en S3*



```
lambda_function.py
1 import requests
2 import boto3
3
4 s3 = boto3.client("s3")
5 base_url = "https://d37ci6vzurychx.cloudfront.net/trip-data/"
6 services = ["yellow", "green", "fhv", "fhvhv"]
7 years = [2024, 2025, 2026]
8
9 def lambda_handler(event, context):
10     bucket = "xideralaws-curso-paoesquivel2026"
11     archivos_subidos = []
12
13     for year in years:
14         for service in services:
15             for m in range(1, 13):
16                 url = f"{base_url}{service}_tripdata_{year}-{str(m).zfill(2)}.parquet"
17                 filename = f"{service}_{year}_{str(m).zfill(2)}.parquet"
18                 key = f"{year}/{filename}"
19
20                 r = requests.get(url)
21                 print(f"{url} -> {r.status_code}")
22
23                 if r.status_code == 200:
24                     s3.put_object(Bucket=bucket, Key=key, Body=r.content)
25                     archivos_subidos.append(key)
26                 else:
27                     print(f"Archivo no disponible: {url}")
28
29     return {
30         "statusCode": 200,
31         "message": "Carga completada en S3",
32         "files": archivos_subidos
33     }
```



```
from pathlib import Path
from pyspark.sql import SparkSession

bucket = "xideralaws-curso-paoesquivel2026"
years = [2024, 2025, 2026]
base_dir = Path.home() / "data" / "nyc_all" / "raw"
base_dir.mkdir(parents=True, exist_ok=True)

s3 = boto3.client("s3")
spark = SparkSession.builder.appName("NYC Data").getOrCreate()

dfs = []

for year in years:
    year_dir = base_dir / str(year)
    year_dir.mkdir(exist_ok=True)

    response = s3.list_objects_v2(Bucket=bucket, Prefix=f"{year}/")
    local_files = []

    if "Contents" in response:
        for obj in response["Contents"]:
            key = obj["Key"]
            if key.endswith(".parquet"):
                local_file = year_dir / Path(key).name
                if local_file.exists() and local_file.stat().st_size > 0:
                    print(f"Ya existe localmente: {local_file}")
                else:
                    print(f"Descargando {key} -> {local_file}")
                    s3.download_file(bucket, key, str(local_file))
                    local_files.append(str(local_file))

    if local_files:
        df_year = spark.read.parquet(*local_files)
        dfs.append(df_year)
```

```
Ya existe localmente: /home/ubuntu/data/nyc_all/raw/2024/fhv_2024_01.parquet
Ya existe localmente: /home/ubuntu/data/nyc_all/raw/2024/fhv_2024_02.parquet
Ya existe localmente: /home/ubuntu/data/nyc_all/raw/2024/fhv_2024_03.parquet
```

# Limpieza y transformación de data

# Limpieza y transformación de data

collapse Output

```
import Path
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

base_dir = Path.home() / "data" / "nyc_all" / "raw"
years = [2024, 2025, 2026]

spark = SparkSession.builder.appName("NYC Taxi Data").getOrCreate()

dfs = []

for year in years:
    year_dir = base_dir / str(year)
    print(f"Leyendo carpeta local: {year_dir}")
    df_year = spark.read.parquet(f"file://{year_dir}")
    df_year = df_year.withColumn("year", F.lit(year))
    dfs.append(df_year)
```

```
Leyendo carpeta local: /home/ubuntu/data/nyc_all/raw/2024
Leyendo carpeta local: /home/ubuntu/data/nyc_all/raw/2025
Leyendo carpeta local: /home/ubuntu/data/nyc_all/raw/2026
```

```
[31]: from pathlib import Path
      from pyspark.sql import SparkSession
      from pyspark.sql import functions as F

      spark = SparkSession.builder.appName("NYC Taxi Data").getOrCreate()

      year_dir = Path.home() / "data" / "nyc_all" / "raw" / "2024"

      # Leer servicio yellow
      df_yellow = spark.read.parquet(f"file://{year_dir}/yellow_2024_*.parquet")
      df_yellow = df_yellow.withColumn("service", F.lit("yellow"))

      # Leer servicio green
      df_green = spark.read.parquet(f"file://{year_dir}/green_2024_*.parquet")
      df_green = df_green.withColumn("service", F.lit("green"))

      # Unir ambos
      df_two = df_yellow.union(df_green)

      df_two.show(5)
```

```
270 else:
271     raise
```

AnalysisException: [NUM\_COLUMNS\_MISMATCH] UNION can only be performed on inputs with the same number of columns, but the first input has 20 columns and the second input has 21 columns. SQLSTATE: 42826;

# Limpieza y transformación de data

## Unión Yellow años 25-26

```
[6]: from pathlib import Path
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("NYC Taxi Data").getOrCreate()

years = [2025, 2026]
meses = [f"{m:02d}" for m in range(1, 13)]
dfs = []

for year in years:
    year_dir = Path.home() / "data" / "nyc_all" / "raw" / str(year)
    for mes in meses:
        file_path = year_dir / f"yellow_{year}_{mes}.parquet"
        if file_path.exists():
            df_mes = spark.read.parquet(f"file://{file_path}") \
                .withColumn("service", F.lit("yellow")) \
                .withColumn("year", F.lit(year)) \
                .withColumn("month", F.lit(mes))

            #if "cbd_congestion_fee" not in df_mes.columns:
            #    df_mes = df_mes.withColumn("cbd_congestion_fee", F.lit(None))

            dfs.append(df_mes)
            print(f"Archivo disponible: {file_path}")
        else:
            print(f"Archivo no disponible: {file_path}")

# Unir todos los DataFrames encontrados
if dfs:
    df_all_yellow = dfs[0]
    for df in dfs[1:]:
        df_all_yellow = df_all_yellow.union(df)
    print("Unión completa")
    df_all_yellow.show(5)
else:
    print("No se encontró ningún archivo en los años/meses especificados")
```

## Unión Green años 25-26

```
[: from pathlib import Path
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("NYC Taxi Data").getOrCreate()

years = [2025, 2026]
meses = [f"{m:02d}" for m in range(1, 13)]
dfs = []

for year in years:
    year_dir = Path.home() / "data" / "nyc_all" / "raw" / str(year)
    for mes in meses:
        file_path = year_dir / f"green_{year}_{mes}.parquet"
        if file_path.exists():
            df_mes = spark.read.parquet(f"file://{file_path}") \
                .withColumn("service", F.lit("green")) \
                .withColumn("year", F.lit(year)) \
                .withColumn("month", F.lit(mes))

            #if "cbd_congestion_fee" not in df_mes.columns:
            #    df_mes = df_mes.withColumn("cbd_congestion_fee", F.lit(None))

            dfs.append(df_mes)
            print(f"Archivo disponible: {file_path}")
        else:
            print(f"Archivo no disponible: {file_path}")

# Unir todos los DataFrames encontrados
if dfs:
    df_all_green = dfs[0]
    for df in dfs[1:]:
        df_all_green = df_all_green.union(df)
    print("Unión completa")
    df_all_green.show(5)
else:
    print("No se encontró ningún archivo en los años/meses especificados")
```

# Limpieza y transformación de data

# Limpieza y transformación de data

### Eliminacion de columnas diferentes en cada servicio

```
[8]: # columna airport_fee es diferente or ende se elimina
df_all_yellow = df_all_yellow.drop("airport_fee")
df_all_yellow.show(5)

#numero de columnas
print("Columnas:", len(df_all_yellow.columns))
```

| VendorID | trip_pickup_datetime | trip_dropoff_datetime | passenger_count | trip_distance | RatecodeID | store_and_fwd_flag | PULocationID | DOLocationID | payment_type | fare_amount | extra | mta_tax | tip_amount | tolls_amount | improvement_surcharge | total_amount | congestion_surcharge | cbd_congestion_fee | service | year | month |
|----------|----------------------|-----------------------|-----------------|---------------|------------|--------------------|--------------|--------------|--------------|-------------|-------|---------|------------|--------------|-----------------------|--------------|----------------------|--------------------|---------|------|-------|
| 1        | 2025-01-01 00:18:38  | 2025-01-01 00:26:59   | 1               | 1.6           | 1          | N                  | 229          | 237          | 1            | 10.0        | 3.5   | 0.5     | 3.0        | 0.0          | 1.0                   | 18.0         | 2.5                  | 0.0                | yellow  | 2025 | 01    |
| 1        | 2025-01-01 00:32:40  | 2025-01-01 00:35:13   | 1               | 0.5           | 1          | N                  | 236          | 237          | 1            | 5.1         | 3.5   | 0.5     | 2.02       | 0.0          | 1.0                   | 12.12        | 2.5                  | 0.0                | yellow  | 2025 | 01    |
| 1        | 2025-01-01 00:44:04  | 2025-01-01 00:46:01   | 1               | 0.6           | 1          | N                  | 141          | 141          | 1            | 5.1         | 3.5   | 0.5     | 2.0        | 0.0          | 1.0                   | 12.1         | 2.5                  | 0.0                | yellow  | 2025 | 01    |
| 2        | 2025-01-01 00:14:27  | 2025-01-01 00:20:01   | 3               | 0.52          | 1          | N                  | 244          | 244          | 2            | 7.2         | 1.0   | 0.5     | 0.0        | 0.0          | 1.0                   | 9.7          | 0.0                  | 0.0                | yellow  | 2025 | 01    |
| 2        | 2025-01-01 00:21:34  | 2025-01-01 00:25:06   | 3               | 0.66          | 1          | N                  | 244          | 116          | 2            | 5.8         | 1.0   | 0.5     | 0.0        | 0.0          | 1.0                   | 8.3          | 0.0                  | 0.0                | yellow  | 2025 | 01    |

only showing top 5 rows  
Columnas: 22

```
[9]: # columna es diferente por ende se elimina ehail_fee & trip_type
df_all_green = df_all_green.drop("ehail_fee")
df_all_green = df_all_green.drop("trip_type")
df_all_green.show(5)
#numero de columnas
print("Columnas:", len(df_all_green.columns))
```

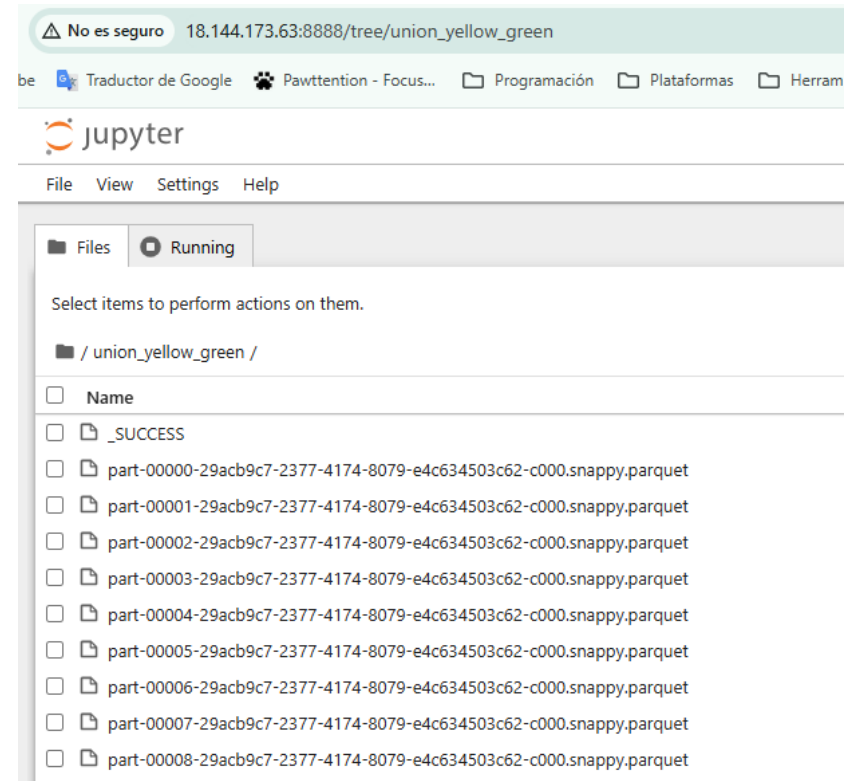
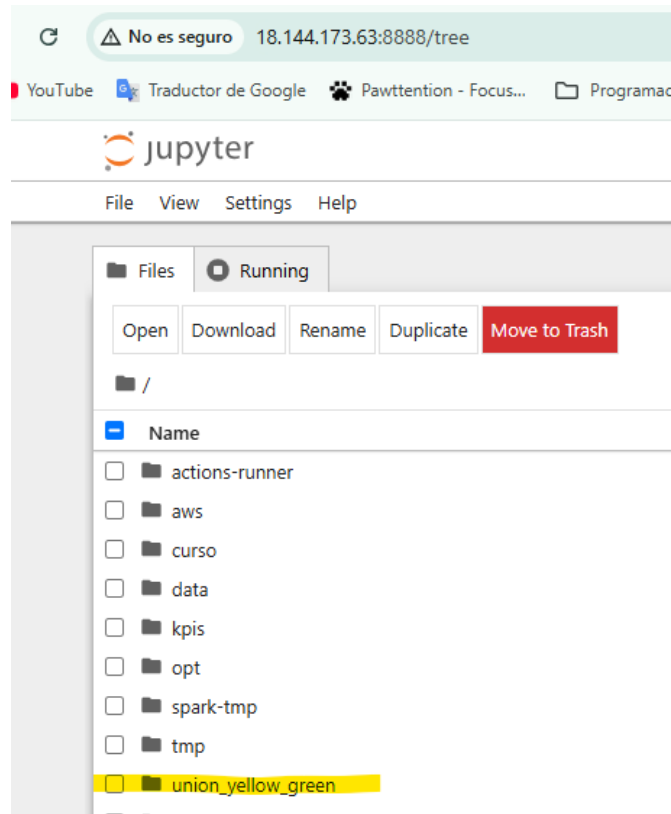
```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```





# Unión dataFrames y guardado

```
# Guardar la unión de yellow y green taxis  
df_all_color_taxi.write.mode("overwrite").parquet("union_yellow_green/")
```



# KPIs Obtenidos

## ▼ KPIs YELLOW & GREEN NYC Taxis

```
[1]: from pyspark.sql import SparkSession
```

```
spark = (  
    SparkSession.builder  
        .appName("TaxiDataLimpieza")  
        .config("spark.driver.memory", "2g") # ✅ seguro para tu instancia  
        .config("spark.sql.shuffle.partitions", "100")  
        .getOrCreate()  
)
```

```
WARNING: Using incubator modules: jdk.incubator.vector  
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
26/07/11 21:07:13 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... usi
```

```
[2]: import glob  
from pyspark.sql import functions as F  
from pathlib import Path
```

```
        .orderBy(F.desc("count"))  
    )  
  
    # --- KPI 2: Monto promedio por servicio ---  
    kpi_monto = (  
        df_sample_all_taxis.groupBy("service")  
        .agg(F.avg("total_amount").alias("avg_total"))  
    )  
  
    # --- KPI 3: Tipo de pago (efectivo vs tarjeta) ---  
    kpi_pago = (  
        df_sample_all_taxis.filter(F.col("payment_type").isin([1, 2]))  
        .withColumn(  
            "payment_desc",  
            F.when(F.col("payment_type") == 1, "Credit Card")  
            .when(F.col("payment_type") == 2, "Cash")  
        )  
        .groupBy("payment_desc")  
        .agg(F.avg("total_amount").alias("avg_total"))  
    )  
  
    # Convertir a pandas para exportar  
    pdf_servicio = kpi_servicio.toPandas()
```

# Exportación de KPIs a s3

```
] import boto3
import json

kpis = {
    "servicio_mas_solicitado": pdf_servicio.to_dict(orient="records"),
    "monto_promedio_servicio": pdf_monto.to_dict(orient="records"),
    "pago_promedio": pdf_pago.to_dict(orient="records")
}
```

*# Guardar localmente primero*

```
with open("kpis_color_taxis_v5.json", "w") as f:
    json.dump(kpis, f, indent=4)
```

*# Subir a S3*

```
s3 = boto3.client("s3")
bucket_name = "xideralaws-curso-paoesquivel2026"
object_key = "kpis_taxis/kpis_color_taxis_v4.json"
```

```
s3.upload_file("kpis_color_taxis_v5.json", bucket_name, object_key)
```

```
print("KPIs subidos a S3:", f"s3://{bucket_name}/{object_key}")
```

KPIs subidos a S3: s3://xideralaws-curso-paoesquivel2026/kpis\_taxis/kpis\_color\_taxis\_v4.json

xideralaws-curso-paoesquivel2026 [Info](#)

[Objects](#)

[Metadata](#)

[Properties](#)

[Permissions](#)

Objects (8)



[Copy S3 URI](#)

[Copy URL](#)

[Download](#)

Objects are the fundamental entities stored in Amazon S3. You can [Learn more](#)



Name



Type



[2024/](#)

Folder



[2025/](#)

Folder



[2026/](#)

Folder



[hola.txt](#)

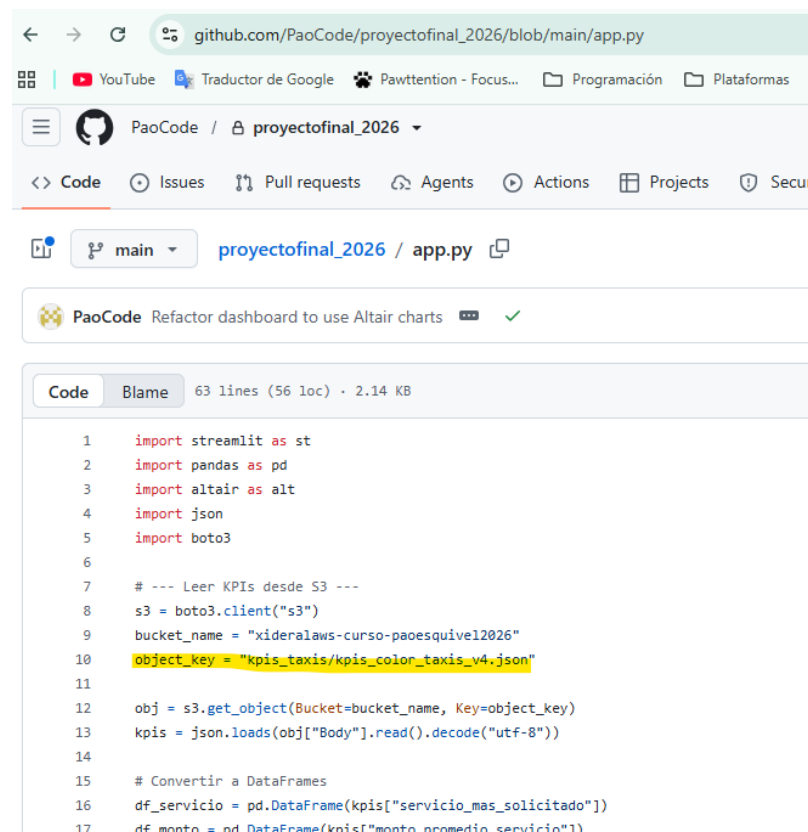
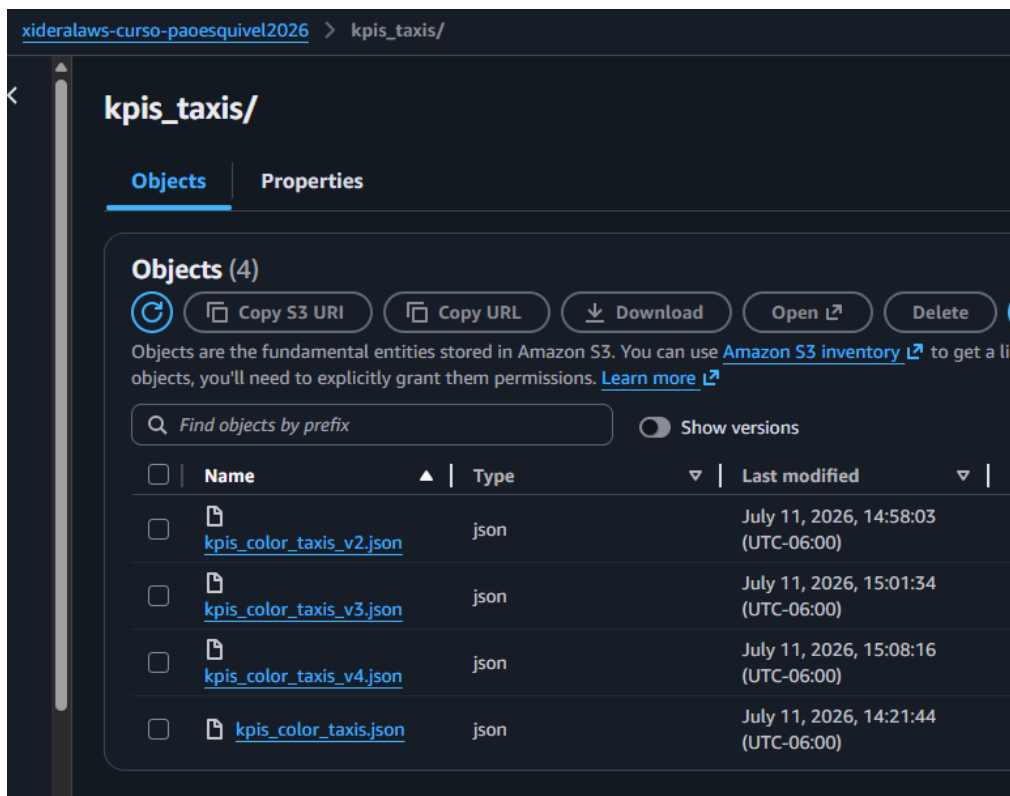
txt



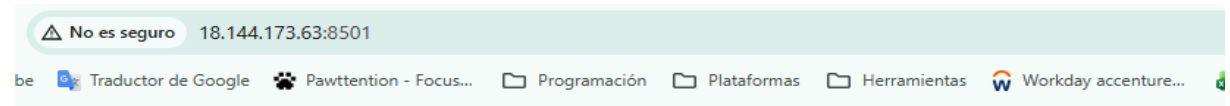
[kpis\\_taxis/](#)

Folder

# Consumo de KPIs por Streamlite

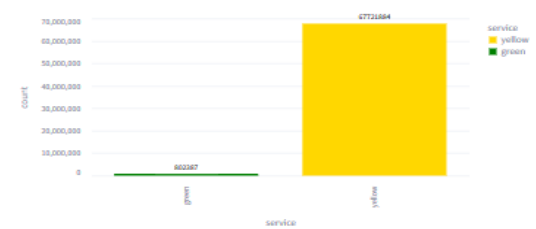


# Despliegue y visualización en Streamlite



## Dashboard KPIs Taxic NYC

### Servicio más solicitado



### Monto promedio por servicio



### Pago promedio por tipo de pago

